Department of Computer Engineering
Faculty of Engineering, Kasetsart University

HW#6: Database Design with UML

From Application Requirements to Relational Schema

Objective

The purpose of this assignment is to design a relational database system based on user requirements using UML database design concepts, and then translate the UML design into a relational schema and SQL implementation.

Application choice: You may continue using the same application/domain from your SQL-III assignment (homework 5#), revise it substantially, or choose a new application if your previous design is not suitable or complete enough for this assignment.

Important: Choose an application that is rich enough to demonstrate the required UML concepts naturally and consistently.

Learning Outcomes

- Analyze application requirements and express them as clear business rules.
- Model classes, associations, multiplicities, association classes, and generalization/specialization using UML.
- Translate a UML class diagram into a relational schema with primary and foreign keys.
- Connect database design decisions to SQL CREATE TABLE statements and constraints.

Submission Requirements

- Submit one PDF containing Parts 1-4 in the order shown below, including your SQL, sample data evidence, query results, and short interpretations.
- Your UML class diagram must be readable and may be created with Draw.io, Lucidchart, or another UML tool.
- Include SQL code as text, not only screenshots.
- Your work must be internally consistent across requirements, UML, relational schema, and SQL.
- You may reuse, revise, or replace the application from SQL-III. No instructor approval is required to change the application.

Part 1 - Application Requirements and Business Rules

Briefly describe the application/database you will use for this assignment. The description should be concise but specific enough to justify the UML design that follows.

- 1.1 **Problem and Motivation**: What problem does your application solve, and why is a database needed?
- 1.2 **Target Users**: Who are the main users of the system?
- 1.3 **Main Data**: What major information must the database store?
- 1.4 **Business Rules**: Write at least five important business rules that describe how the data is related or constrained.

> ***Example business rules**: A student can enroll in many courses. A course can contain many students. Each enrollment records a semester and grade. A course is offered by one department. A graduate student may have an advisor.*

Part 2 - Design the UML Class Diagram

Create one UML class diagram that represents the requirements from Part 1. Your diagram must include the following concepts:

- **Classes** with relevant attributes and a clear identifier for each main class.
- **Associations** between classes.
- **Multiplicity** such as 1, 0..1, *, and 1..*.
- At least **one many-to-many relationship**.
- At least **one Association Class** with attributes that belong to the relationship.
- At least one **Superclass-Subclass** structure (generalization/specialization).

Composition and Aggregation: Use them only when they correctly represent your application. They are not required if they do not fit the domain.

Short Explanations

- Q1. **Multiplicity**: Select one association in your diagram and explain its multiplicity in plain language.
- Q2. **Association Class**: Identify your association class. Explain why its attributes belong to the relationship rather than to either participating class.
- Q3. **Generalization**: Explain the meaning of your superclass-subclass relationship and why the subclasses are useful.

Part 3 - Convert UML to a Relational Schema

Convert your UML class diagram into a relational database schema. Show all relations clearly and identify primary keys (PK) and foreign keys (FK).

Example notation:

```
STUDENT(studentID PK, name, major)

COURSE(courseID PK, courseName, credits)

ENROLLMENT(studentID PK/FK, courseID PK/FK, semester PK, grade)
```

- Q4. Choose one **one-to-many association** from your UML diagram and explain how it is represented using a foreign key.
- Q5. Explain how your **many-to-many relationship** or **association class** is converted into relations. Identify the resulting primary and foreign keys.
- Q6. Explain how your **superclass and subclasses** are represented in the relational model.

Connection to SQL-III: This part reuses the SQL constraint concepts practiced previously. You do not need to implement the entire database if it contains more than four relations.

Part 4 - SQL Implementation, Sample Data & Business Queries

Implement a meaningful subset of your relational schema in MySQL. This part connects your UML design to the SQL concepts learned in SQL-I, SQL-II, and SQL-III.

4.1 Create Tables

Select at least four related relations from Part 3 and write SQL CREATE TABLE statements for them. Your implementation must be consistent with the UML diagram and relational schema.

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- At least one additional constraint: UNIQUE, CHECK, or DEFAULT

4.2 Create Sample Data

Insert realistic sample data so that your database can produce meaningful query results. Avoid meaningless values such as A, B, X1, or test1 unless they are appropriate to the domain.

- Insert at least 10 records into each table implemented in Part 4.
- For a relationship/transaction table (for example ENROLLMENT, ORDER_ITEM, BOOKING, or WORKS_ON), include enough records to demonstrate different relationships and query results.
- The data should include some variation (for example different categories, dates, statuses, amounts, or user types) so that aggregation and filtering queries produce meaningful results.

4.3 Answer Interesting Business Questions with SQL

Write at least five SQL queries that answer useful questions about your application. The goal is not simply to demonstrate SQL syntax; each query should provide information that could help a user, manager, or organization understand or operate the application.

Your five queries must collectively include:

- At least one query that **joins multiple tables**.
- At least one query using **GROUP BY** with an aggregate function such as COUNT, SUM, AVG, MIN, or MAX.
- At least one query using **HAVING or a subquery**.
- At least one SQL-III concept such as **CASE**, a **date function**, or a **VIEW**.
- At least one query of your own design that you consider especially useful or interesting for the application.

For each query, present all four items below:

1. Business Question - What useful question are you trying to answer?
2. SQL Query - The SQL statement used to answer it.
3. Result - Show the query output (table or readable screenshot).

Examples of Business Questions

- Which products/categories generate the highest total sales?
- Which customers have made more than three purchases this semester?
- Which courses have enrollment above the average enrollment?
- Which employees are responsible for the largest number of completed jobs?
- How does monthly activity change over time?
- Which records should be classified as High / Medium / Low using a meaningful business rule?

These are examples only. Your questions should fit your own application and data.

Final Consistency Checklist

☐ Every important class in the UML diagram appears in the relational schema.
☐ Association multiplicities are consistent with primary/foreign key placement.
☐ The association class is correctly mapped to a relation.
☐ The superclass-subclass mapping is explained and implemented consistently.
☐ The SQL CREATE TABLE statements match the keys and constraints shown in Part 3.
☐ Sample data are realistic and sufficient to produce meaningful query results.
☐ At least five business questions are answered using SQL.
☐ Each business query includes the question, SQL, and result.

How This Homework Connects to the Course

Requirements → UML → Relations → SQL → Data → Business Questions